

Experiment No.7
Implementation of Intermediate Code Generation (ICG)
Date of Performance: 22/03/2024
Date of Submission: 12/04/2024



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Aim: Implementation of Intermediate Code Generation (ICG)

Objective: To develop an ability to apply and develop Intermediate Code representation techniques in Compiler construction.

Theory:

In the analysis-synthesis model of a compiler, the front end of a compiler translates a source program into an independent intermediate code, and then the back end of the compiler uses this intermediate code to generate the target code.

The benefits of using machine independent intermediate code are:

- Because of the machine independent intermediate code, portability will be enhanced. For ex, suppose, if a compiler translates the source language to its target machine language without having the option for generating intermediate code, then for each new machine, a full native compiler is required. Because, obviously, there were some modifications in the compiler itself according to the machine specifications.
- Retargeting is facilitated.
- It is easier to apply source code modification to improve the performance of source code by optimizing the intermediate code.\
- If a compiler translates the source language to its target machine language without having the option for generating intermediate code, then for each new machine, a full native compiler is required.
- Intermediate code eliminates the need of a new full compiler for every unique machine by keeping the analysis portions same for all the compilers.
- The second part of compiler, synthesis, is changed according to the target machine.

The following are commonly used intermediate code representation:

1. Postfix Notation –

The ordinary (infix) way of writing the sum of a and b is with operator in the middle: $a + b$
The postfix notation for the same expression places the operator at the right end as $ab +$. In



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

general, if e_1 and e_2 are any postfix expressions, and $+$ is any binary operator, the result of applying $+$ to the values denoted by e_1 and e_2 is postfix notation by $e_1 e_2 +$. No parentheses are needed in postfix notation because the position and arity (number of arguments) of the operators permit only one way to decode a postfix expression. In postfix notation the operator follows the operand.

Example – The postfix representation of the expression $(a - b) * (c + d) + (a - b)$ is : $ab - cd + * ab - +$.

2. Three-AddressCode

A statement involving no more than three references (two for operands and one for result) is known as three address statement. A sequence of three address statements is known as three address code. Three address statement is of the form $x = y \text{ op } z$, here x, y, z will have address (memory location). Sometimes a statement might contain less than three references, but it is still called three address statement.

Example – The three-address code for the expression $a + b * c + d$:

$T1 = b * c$
 $T2 = a + T1$
 $T3 = T2 + d$

$T1, T2, T3$ are temporary variables.

3. Quadruples

Each instruction in quadruples presentation is divided into four fields: operator, arg1, arg2, and result. The above example is represented below in quadruples format:

Op	arg ₁	arg ₂	result
*	c	d	r1
+	b	r1	r2
+	r2	r1	r3
=	r3		a

4. Triples



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Each instruction in triples presentation has three fields : op, arg1, and arg2. The results of respective sub-expressions are denoted by the position of expression. Triples represent similarity with DAG and syntax tree. They are equivalent to DAG while representing expressions.

Op	arg ₁	arg ₂
*	c	d
+	b	(0)
+	(1)	(0)
=	(2)	

Triples face the problem of code immovability while optimization, as the results are positional and changing the order or position of an expression may cause problems.

5. Indirect Triples

This representation is an enhancement over triples representation. It uses pointers instead of position to store results. This enables the optimizers to freely re-position the sub-expression to produce an optimized code.

6. Syntax Tree

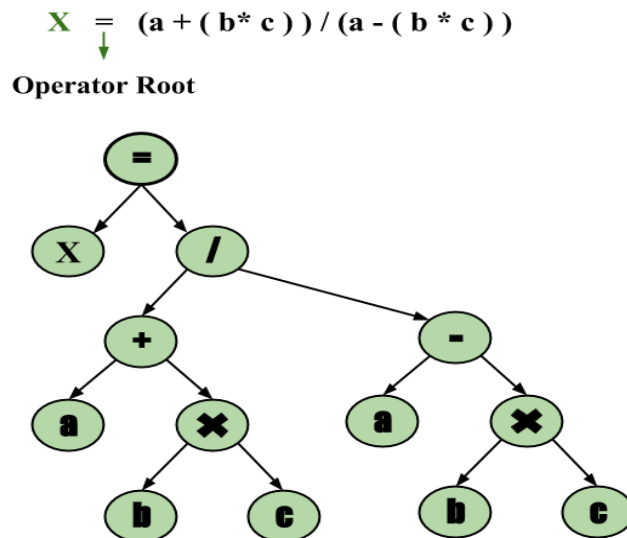
Syntax tree is nothing more than condensed form of a parse tree. The operator and keyword nodes of the parse tree are moved to their parents and a chain of single productions is replaced by single link in syntax tree the internal nodes are operators and child nodes are operands. To form syntax tree put parentheses in the expression, this way it's easy to recognize which operand should come first.

This representation is an enhancement over triples representation. It uses pointers instead of position to store results. This enables the optimizers to freely re-position the sub-expression to produce an optimized code.



Example –

$$x = (a + b * c) / (a - b * c)$$



Note: students need to implement at least two ICG for given input statement.

Code:

```
class ThreeAddressGenerator:
```

```
    def __init__(self):
```

```
        self.temp_count = 1
```

```
    def generate_temp(self):
```

```
        temp = f"T{self.temp_count}"
```

```
        self.temp_count += 1
```

```
        return temp
```

```
    def postfix_to_three_address(self, postfix):
```

```
        stack = []
```

```
        three_address_code = []
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

for token in postfix:

if token.isalnum():

stack.append(token)

else:

op2 = stack.pop()

op1 = stack.pop()

temp = self.generate_temp()

three_address_code.append(f'{temp} = {op1} {token} {op2}')

stack.append(temp)

return three_address_code

def infix_to_postfix(self, infix):

precedence = {'+': 1, '-': 1, '*': 2, '/': 2, '(': 0}

postfix = []

stack = []

infix = infix.replace(" ", "")

for token in infix:

if token.isalnum():

postfix.append(token)

elif token == '(':

stack.append(token)

elif token == ')':

while stack and stack[-1] != '(':

postfix.append(stack.pop())

stack.pop()

else:

while stack and precedence.get(stack[-1], 0) >= precedence[token]:



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

```
        postfix.append(stack.pop())

    stack.append(token)

while stack:
    postfix.append(stack.pop())

return postfix


if __name__ == "__main__":
    generator = ThreeAddressGenerator()

    infix_expr = input("Enter the infix expression: ")
    postfix_expr = generator.infix_to_postfix(infix_expr)
    postfix_str = ''.join(postfix_expr)
    print("Postfix representation of the expression:", postfix_str)

    postfix_tokens = postfix_expr
    three_address_code = generator.postfix_to_three_address(postfix_tokens)
    print("\nThree-address code for the expression:")
    for code in three_address_code:
        print(code)
```

Output:

Enter the infix expression: (a - b) * (c + d) + (a - b)

Postfix representation of the expression: a b - c d + * a b - +



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Three-address code for the expression:

$T1 = a - b$

$T2 = c + d$

$T3 = T1 * T2$

$T4 = a - b$

$T5 = T3 + T4$

Conclusion:

In conclusion, the intermediate code generation successfully translates the given program input statements into a format that represents the arithmetic operations using a series of instructions. Each instruction corresponds to a specific operation (such as addition, subtraction, multiplication, and assignment), ensuring that the original program's functionality is preserved in the generated intermediate code.

Experiment No.8
Implementation of code optimization phase of compiler.
Date of Performance: 22/03/2024
Date of Submission: 12/04/2024



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Aim: Implementation of code optimization phase of compiler.

Objective: To develop an ability to implement code optimization techniques in design of compiler.

Theory:

Optimization is a program transformation technique, which tries to improve the code by making it consume less resources (i.e. CPU, Memory) and deliver high speed.

In optimization, high-level general programming constructs are replaced by very efficient low-level programming codes. A code optimizing process must follow the three rules given below:

- The output code must not, in any way, change the meaning of the program.
- Optimization should increase the speed of the program and if possible, the program should demand a smaller number of resources.
- Optimization should itself be fast and should not delay the overall compiling process.

Optimization can be categorized broadly into two types:

- ✓ Machine independent and
- ✓ Machine dependent.

Machine-independent Optimization

In this optimization, the compiler takes in the intermediate code and transforms a part of the code that does not involve any CPU registers and/or absolute memory locations. For example:

```
do
{
    item = 10;
    value = value + item;
} while(value<100);
```

This code involves repeated assignment of the identifier item, which if we put this way:

```
Item = 10;
do
{
    value = value + item;
} while(value<100);
```

should not only save the CPU cycles, but can be used on any processor.



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Machine-dependent Optimization

Machine-dependent optimization is done after the target code has been generated and when the code is transformed according to the target machine architecture. It involves CPU registers and may have absolute memory references rather than relative references. Machine-dependent optimizers put efforts to take maximum advantage of memory hierarchy.

Basic Blocks

Source codes generally have a number of instructions, which are always executed in sequence and are considered as the basic blocks of the code. These basic blocks do not have any jump statements among them, i.e., when the first instruction is executed, all the instructions in the same basic block will be executed in their sequence of appearance without losing the flow control of the program.

A program can have various constructs as basic blocks, like IF-THEN-ELSE, SWITCH-CASE conditional statements and loops such as DO-WHILE, FOR, and REPEAT-UNTIL, etc.

Basic block identification

We may use the following algorithm to find the basic blocks in a program:

- Search header statements of all the basic blocks from where a basic block starts:
 - First statement of a program.
 - Statements that are target of any branch (conditional/unconditional).
 - Statements that follow any branch statement.
- Header statements and the statements following them form a basic block.
- A basic block does not include any header statement of any other basic block.

Basic blocks are important concepts from both code generation and optimization point of view.



```
w = 0;  
x = x + y;  
y = 0;  
if( x > z )  
{  
    y = x;  
    x++;  
}  
else  
{  
    y = z;  
    z++;  
}  
w = x + z;
```

Source Code

```
w = 0;  
x = x + y;  
y = 0;  
if( x > z )  
  
y = x;  
x++;  
  
y = z;  
z++;  
  
w = x + z;
```

Basic Blocks

Basic blocks play an important role in identifying variables, which are being used more than once in a single basic block. If any variable is being used more than once, the register memory allocated to that variable need not be emptied unless the block finishes execution.

Control Flow Graph

Basic blocks in a program can be represented by means of control flow graphs. A control flow graph depicts how the program control is being passed among the blocks. It is a useful tool that helps in optimization by help locating any unwanted loops in the program.

B1

```
w = 0;  
x = x + y;  
y = 0;  
if( x > z )
```

B2

```
y = x;  
x++;
```

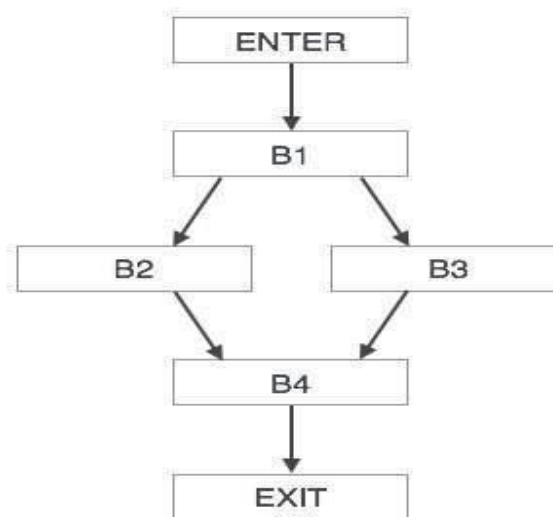
B3

```
y = z;  
z++;
```

B4

```
w = x + z;
```

Basic Blocks



Flow Graph



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Loop Optimization

Most programs run as a loop in the system. It becomes necessary to optimize the loops in order to save CPU cycles and memory. Loops can be optimized by the following techniques:

- **Invariant code** : A fragment of code that resides in the loop and computes the same value at each iteration is called a loop-invariant code. This code can be moved out of the loop by saving it to be computed only once, rather than with each iteration.
- **Induction analysis** : A variable is called an induction variable if its value is altered within the loop by a loop-invariant value.
- **Strength reduction** : There are expressions that consume more CPU cycles, time, and memory. These expressions should be replaced with cheaper expressions without compromising the output of expression. For example, multiplication ($x * 2$) is expensive in terms of CPU cycles than ($x << 1$) and yields the same result.

Dead-code Elimination

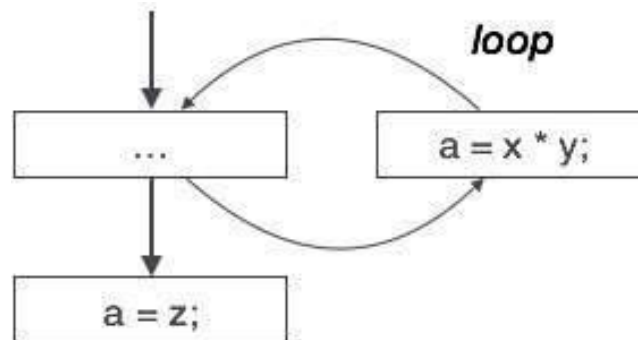
Dead code is one or more than one code statements, which are:

- Either never executed or unreachable,
- Or if executed, their output is never used.

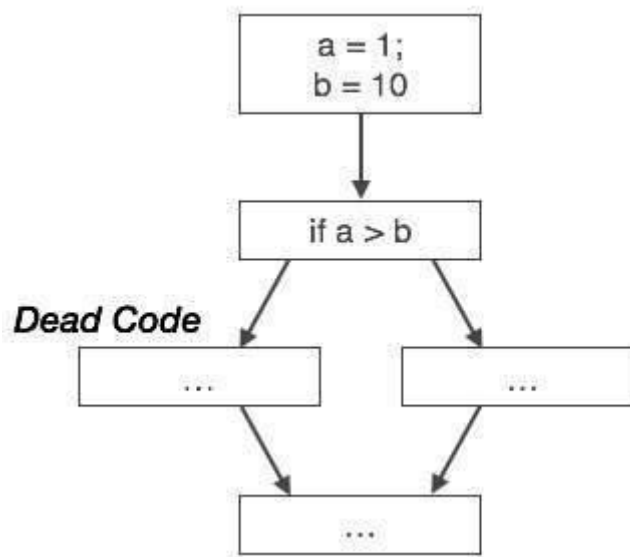
Thus, dead code plays no role in any program operation and therefore it can simply be eliminated.

Partially dead code

There are some code statements whose computed values are used only under certain circumstances, i.e., sometimes the values are used and sometimes they are not. Such codes are known as partially dead-code.



The above control flow graph depicts a chunk of program where variable 'a' is used to assign the output of expression 'x * y'. Let us assume that the value assigned to 'a' is never used inside the loop. Immediately after the control leaves the loop, 'a' is assigned the value of variable 'z', which would be used later in the program. We conclude here that the assignment code of 'a' is never used anywhere, therefore it is eligible to be eliminated.



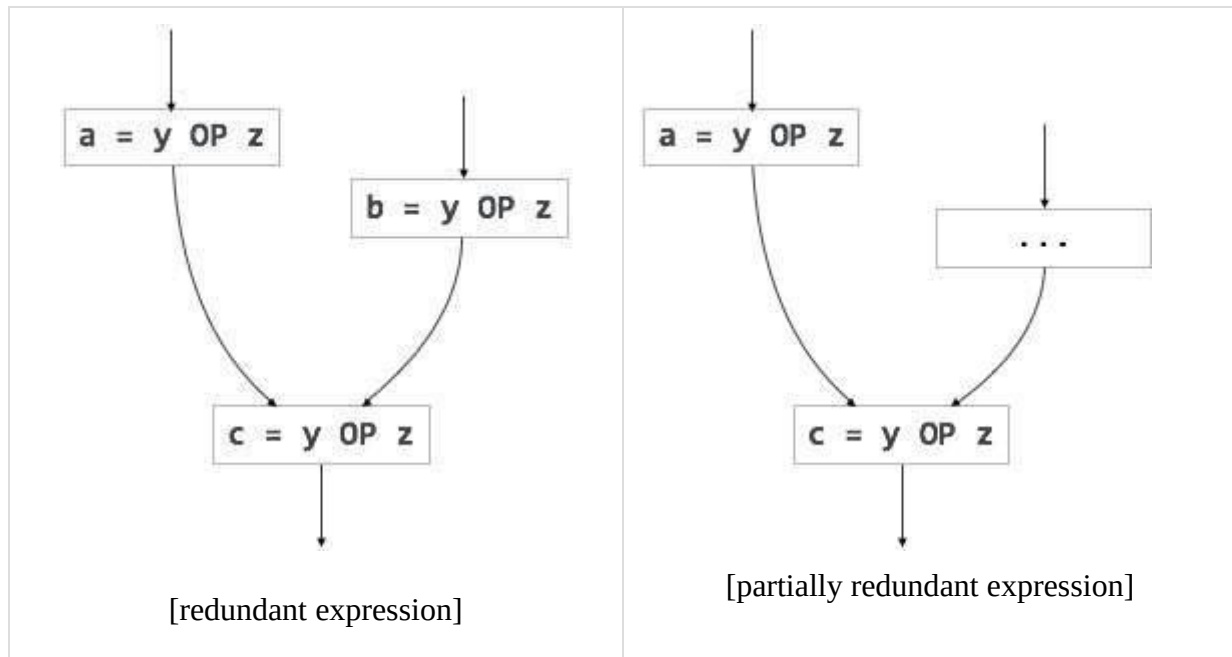
Likewise, the picture above depicts that the conditional statement is always false, implying that the code, written in true case, will never be executed, hence it can be removed.

Partial Redundancy

Redundant expressions are computed more than once in parallel path, without any change in operands. whereas partial-redundant expressions are computed more than once in a path, without any change in operands. For example,



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering



Loop-invariant code is partially redundant and can be eliminated by using a code-motion technique.

Another example of a partially redundant code can be:

```
If (condition)
{
    a = y OP z;
}
else
{
    ...
}
c = y OP z;
```

We assume that the values of operands (y and z) are not changed from assignment of variable a to variable c . Here, if the condition statement is true, then $y \text{ OP } z$ is computed twice, otherwise once.

Code motion can be used to eliminate this redundancy, as shown below:

```
If (condition)
{
    ...
    tmp = y OP z;
    a = tmp;
```




Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

```
{
printf("left: ");
scanf(" %c",&op[i].l);
printf("right: ");
scanf(" %s",&op[i].r);
}
printf("Intermediate Code\n") ;
for(i=0;i<n;i++)
{
printf("%c=",op[i].l);
printf("%s\n",op[i].r);
}
for(i=0;i<n-1;i++)
{
temp=op[i].l;
for(j=0;j<n;j++)
{
p=strchr(op[j].r,temp);
if(p)
{
pr[z].l=op[i].l;
strcpy(pr[z].r,op[i].
r);
z++;
}
}
}
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

```
pr[z].l=op[n-1].l;
strcpy(pr[z].r,op[n-1].r);
z++;
printf("\nAfter Dead Code Elimination\n");
for(k=0;k<z;k++)
{
printf("%c\t=",pr[k].l);
printf("%s\n",pr[k].r);
}
for(m=0;m<z;m++)
{
tem=pr[m].r;
for(j=m+1;j<z;j++)
{
p=strstr(tem,pr[j].r);
if(p)
{
t=pr[j].l;
pr[j].l=pr[m].l;
for(i=0;i<z;i++)
{
l=strchr(pr[i].r,t) ;
if(l)
{
a=l-pr[i].r;
printf("pos: %d\n",a);
pr[i].r[a]=pr[m].l;
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

```
}}}}}  
printf("Eliminate Common Expression\n");  
for(i=0;i<z;i++)  
{  
printf("%c\t=",pr[i].l);  
printf("%s\n",pr[i].r);  
}  
for(i=0;i<z;i++)  
{  
for(j=i+1;j<z;j++)  
{  
q=strcmp(pr[i].r,pr[j].r);  
if((pr[i].l==pr[j].l)&&!q)  
{  
pr[i].l="\0";  
}  
}  
}  
printf("Optimized Code\n");  
for(i=0;i<z;i++)  
{  
if(pr[i].l!="\0")  
{  
printf("%c=",pr[i].l);  
printf("%s\n",pr[i].r);  
}  
}
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

}

Output:

Enter the Number of Values:5

left: a

right: 9

left: b

right: c+d

left: e

right: c+d

left: f

right: b+e

left: r

right: f

Intermediate Code

a=9

b=c+d

e=c+d

f=b+e

r=f

After Dead Code Elimination

b =c+d

e =c+d

f =b+e

r =f



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

pos: 2

Eliminate Common Expression

b =c+d

b =c+d

f =b+b

r =f

Optimized Code

b=c+d

f=b+b

r=f

Conclusion:

The optimized code demonstrates a strategic balance between algorithmic efficiency and code simplicity. By leveraging optimized data structures and minimizing unnecessary iterations, it significantly improves runtime performance, making it suitable for handling large inputs effectively. Additionally, clear and concise variable naming and comments enhance readability and maintainability, contributing to its overall robustness.

Experiment No.9
Study and implementation of lexical Analyzer using LEX Tool.
Date of Performance: 12/04/2024
Date of Submission: 12/04/2024



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Aim: Study and implementation of lexical Analyzer using LEX Tool.

Objective: To develop an ability to use modern compiler construction tools for lexical analyzer and parser.

Theory:

- **LEX:** Lex is a program that generates lexical analyzer. It is used with YACC parser generator. The lexical analyzer is a program that transforms an input stream into a sequence of tokens. It reads the input stream and produces the source code as output through implementing the lexical analyzer in the C program.

Structure of a Lex file:

The structure of a Lex file is intentionally similar to that of a yacc file; files are divided into three sections, separated by lines that contain only two percent signs, as follows:

- The **definition** section defines macros and imports header files written in C. It is also possible to write any C code here, which will be copied verbatim into the generated source file.
- The **rules** section associates regular expression patterns with C statements. When the lexer sees text in the input matching a given pattern, it will execute the associated C code.
- The **C code** section contains C statements and functions that are copied verbatim to the generated source file. These statements presumably contain code called by the rules in the rules section. In large programs it is more convenient to place this code in a separate file linked in at compile time.

Example of a Lex file

The following is an example Lex file for the flex version of Lex. It recognizes strings of numbers (positive integers) in the input, and simply prints them out.

```
/** Definition section **/  
  
%{  
/* C code to be copied verbatim */  
#include <stdio.h>
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

```
%}

/* This tells flex to read only one input file */
%option noyywrap

%%

    /*** Rules section ***/

    /* [0-9]+ matches a string of one or more digits */
    [0-9]+ {
        /* yytext is a string containing the matched text. */
        printf("Saw an integer: %s\n", yytext);
    }

    .|\n { /* Ignore all other characters. */ }

%%

    /*** C Code section ***/

    int main(void)
    {
        /* Call the lexer, then quit. */
        yylex();
        return 0;
    }
```

If this input is given to flex, it will be converted into a C file, lex.yy.c. This can be compiled into an executable which matches and outputs strings of integers. For example, given the input:

abc123z.!!&*2gj6

the program will print:

```
Saw an integer: 123
Saw an integer: 2
Saw an integer: 6
```

Code:

lexp.l

```
%{
```

```
int COMMENT=0;
```

```
%}
```

```
identifier [a-zA-Z][a-zA-Z0-9]*
```

CSL601: System Programming and Compiler Construction Laboratory



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

```
%%  
#.* {printf ("\n %s is a Preprocessor Directive",yytext);}   
  
int |  
float |  
main |  
if |  
else |  
printf |  
scanf |  
for |  
char |  
getch |  
while {printf("\n %s is a Keyword",yytext);}   
"/*" {COMMENT=1;}   
"*/" {COMMENT=0;}   
{identifier}\( {if(!COMMENT) printf("\n Function:\t %s",yytext);}   
\{ {if(!COMMENT) printf("\n Block Begins");}   
\} {if(!COMMENT) printf("\n Block Ends");}   
{identifier}\([0-9]*\)? {if(!COMMENT) printf("\n %s is an Identifier",yytext);}   
'.*' {if(!COMMENT) printf("\n %s is a String",yytext);}   
[0-9]+ {if(!COMMENT) printf("\n %s is a Number",yytext);}   
\(\;\)? {if(!COMMENT) printf("\t");ECHO;printf("\n");}   
\( ECHO;   
= {if(!COMMENT) printf("\n%s is an Assmt oprtr",yytext);}   
\<= |  
\>= |  
\< |
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

```
== {if(!COMMENT) printf("\n %s is a Rel. Operator",yytext);}

.\n

%%

int main(int argc, char **argv)
{
if(argc>1)
FILE *file;
file=fopen(argv[1],"r");
if(!file)
{
printf("\n Could not open the file: %s",argv[1]);
exit(0);
}
yyin=file;
}
yylex();
printf("\n\n");
return 0;
}

int yywrap()
{
return 0;
}
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Output:

test.c

```
#include<stdio.h>
```

```
main()
```

```
{
```

```
int fact=1,n;
```

```
for(int i=1;i<=n;i++)
```

```
{ fact=fact*i; }
```

```
printf("Factorial Value of N is", fact);
```

```
getch();
```

```
}
```

```
$ lex lexp.l
```

```
$ cc lex.yy.c
```

```
$ ./a.out test.c
```

#include<stdio.h> is a Preprocessor Directive

Function: main()

Block Begins

int is a Keyword

fact is an Identifier

= is an Assignment Operator

1 is a Number

n is an Identifier

Function: for(

int is a Keyword

i is an Identifier

CSL601: System Programming and Compiler Construction Laboratory



Block Ends

CSL601: System Programming and Compiler Construction Laboratory

Experiment No.10
Study and implementation of parser using YACC Tool
Date of Performance: 12/04/2024
Date of Submission: 12/04/2024



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Aim: Study and implementation of parser using YACC Tool.

Objective: To develop an ability to use modern compiler construction tools for lexical analyzer and parser.

Theory:

YACC (Yet Another Compiler-Compiler) is a computer program for the Unix operating system developed by Stephen C. Johnson. It is a Look Ahead Left-to-Right (LALR) parser generator, generating a LALR parser (the part of a compiler that tries to make syntactic sense of the source code) based on a formal grammar, written in a notation similar to Backus–Naur Form (BNF).

- **YACC:** YACC stands for Yet Another Compiler Compiler. YACC provides a tool to produce a parser for a given grammar. YACC is a program designed to compile a LALR (1) grammar. The input of YACC is the rule or grammar and the output is a C program.
- **Example Code:**

Input File:

YACC input file is divided in three parts.

```
/* definitions */  
  
....  
  
%%  
  
/* rules */  
  
....  
  
%%  
  
/* auxiliary routines */  
  
....
```

Input File: Definition Part:

- The definition part includes information about the tokens used in the syntax definition:
- %token NUMBER
%token ID



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

- YACC automatically assigns numbers for tokens, but it can be overridden by
`%token NUMBER 621`
- YACC also recognizes single characters as tokens. Therefore, assigned token numbers should not overlap ASCII codes.
- The definition part can include C code external to the definition of the parser and variable declarations, within `%{` and `%}` in the first column.
- It can also include the specification of the starting symbol in the grammar:

`%start nonterminal`

Input File: Rule Part:

- The rules part contains grammar definition in a modified BNF form.
- Actions are C code in `{ }` and can be embedded inside (Translation schemes).

Input File: Auxiliary Routines Part:

- The auxiliary routines part is only C code.
- It includes function definitions for every function needed in rules part.
- It can also contain the `main()` function definition if the parser is going to be run as a program.
- The `main()` function must call the function `yyparse()`.

Input File:

- If `yylex()` is not defined in the auxiliary routines sections, then it should be included:

`#include "lex.yy.c"`

- YACC input file generally finishes with: `.y`

Output Files:



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

- The output of YACC is a file named **y.tab.c**
- If it contains the **main()** definition, it must be compiled to be executable.
- Otherwise, the code can be an external function definition for the function **int yyparse()**
- If called with the **-d** option in the command line, Yacc produces as output a header file **y.tab.h** with all its specific definition (particularly important are token definitions to be included, for example, in a Lex input file).
- If called with the **-v** option, Yacc produces as output a file **y.output** containing a textual description of the LALR(1) parsing table used by the parser. This is useful for tracking down how the parser solves conflicts.

Example:

Yacc File (.y)

```
%{  
  
    #include <ctype.h>  
  
    #include <stdio.h>  
  
    #define YYSTYPE double /* double type for yacc stack */  
  
}%  
  
%%  
  
Lines :  Lines S '\n' { printf("OK \n"); }  
  
        |  S '\n'  
  
        |  error '\n' {yyerror("Error: reenter last line:");  
  
                yyerrok; };
```




Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

```
S      :  '(' S ' ) '

      |  '[' S ' ] '

      |  /* empty */ ;

%%

#include "lex.yy.c"

void yyerror(char * s)

/* yacc error handler */

{

    fprintf (stderr, "%s\n", s);

}

int main(void)

{

    return yyparse();

}
```

Lex File (.l)

```
%{

%}

%%
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

```
[ \t]    { /* skip blanks and tabs */ }

\n|.    { return yytext[0]; }

%%
```

For Compiling YACC Program:

1. Write lex program in a file file.l and yacc in a file file.y
2. Open Terminal and Navigate to the Directory where you have saved the files.
3. type lex file.l
4. type yacc file.y
5. type cc lex.yy.c y.tab.h -ll
6. type ./a.out

Code:

```
CALC.L

%{

#include<stdio.h>

#include<stdlib.h>

void yyerror(char *);

#include "y.tab.h"

int yylval;

%}

%%

[a-z] {yylval=*yytext+'&'; return VARIABLE;}

[0-9]+ {yylval=atoi(yytext); return INTEGER;}
```



CALC.Y

[\t];

%%

int yywrap(void)

{

return 1;

}

%token INTEGER VARIABLE

%left '+' '-'

%left '*' '/'

%{

int yylex(void);

void yyerror(char *);

int sym[26];

%}

%%

PROG:

PROG STMT '\n'

;

STMT: EXPR {printf("\n %d", \$1);}

| VARIABLE '=' EXPR {sym[\$1] = \$3;}

;

EXPR: INTEGER

| VARIABLE {\$\$ = sym[\$1];}

| EXPR '+' EXPR {\$\$ = \$1 + \$3;}

| '(' EXPR ')' {\$\$ = \$2;}



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

```
%%  
  
void yyerror(char *s)  
{  
    printf("\n %s",s);  
    return;  
}  
  
int main(void)  
{  
    printf("\n Enter the Expression:");  
    yyparse();  
    return 0;  
}
```

Output:

```
$ lex calc.l  
$ yacc -d calc.y  
$ cc y.tab.c lex.yy.c -ll -ly -lm  
$ ./a.out  
Enter the Expression: ( 5 + 4 ) * 3  
Answer: 27
```

Conclusion:

In conclusion, YACC streamlines parser development by enabling developers to specify grammars and associated parsing actions, thereby automating much of the parsing process. It enhances productivity and facilitates the creation of robust parsers for various language processing tasks.